O **Git** é um sistema de controle de versão distribuído, usado para rastrear mudanças em arquivos e facilitar a colaboração entre desenvolvedores. Ele permite que múltiplas pessoas trabalhem no mesmo projeto sem conflitos, mantendo um histórico de alterações e possibilitando a reversão de versões anteriores quando necessário.

Fluxo de Trabalho do Git

O fluxo de trabalho básico do Git segue os seguintes passos:

- **Inicialização ou Clonagem do Repositório**
  - Criar um novo repositório: git init
  - Clonar um repositório existente: git clone <URL_DO_REPO>

- **Modificação dos Arquivos**
  - O desenvolvedor edita, adiciona ou remove arquivos no diretório do projeto.

- **Staging (Indexação das Mudanças)**
  - Adicionar arquivos para o "staging area": git add <arquivo>
  - Para adicionar todas as mudanças: git add .

- **Commit (Salvar as Alterações no Histórico)**
  - Criar um commit com uma mensagem descritiva: git commit -m "Descrição das alterações”

- **Envio para o Repositório Remoto**
  - Enviar as mudanças para um repositório remoto: git push origin <branch>

- **Sincronização com o Repositório Remoto**
  - Trazer atualizações do repositório remoto para o local: git pull origin <branch>

- **Gerenciamento de Branches (Ramificações)**
  - Criar uma nova branch: git branch <nome-da-branch>
  - Mudar para uma branch existente: git checkout <nome-da-branch>
  - Criar e mudar para uma branch ao mesmo tempo: git checkout -b <nome-da-branch>
  - Mesclar uma branch na branch atual: git merge <nome-da-branch>

- **Resolução de Conflitos**
  - Quando há conflitos entre mudanças locais e remotas, o Git permite que o desenvolvedor resolva manualmente antes de finalizar a fusão.

Fluxo de Trabalho Padrão:

- Clonar o repositório (git clone)
- Criar uma nova branch (git checkout -b feature-x)
- Modificar arquivos e adicionar ao staging (git add .)
- Fazer commits das mudanças (git commit -m "Descrição")
- Enviar as mudanças para o repositório remoto (git push origin feature-x)
- Criar um **Pull Request (PR)** e revisar o código antes de integrar com a branch principal
- Mesclar a branch (git merge feature-x ou via PR)
- Excluir a branch antiga (git branch -d feature-x)

Esse fluxo facilita a colaboração, permitindo que diferentes desenvolvedores trabalhem simultaneamente sem sobrescrever o código uns dos outros.

1. Repositório
Um **repositório** no Git é um diretório que contém todos os arquivos do projeto e o histórico de versões. Ele pode ser local (no seu computador) ou remoto (em um servidor como GitHub, GitLab ou Bitbucket).

📌 **Exemplo prático:**
Criando um novo repositório e inicializando o Git nele:

- mkdir meu-projeto
- cd meu-projeto
- git init

Isso cria um repositório Git no diretório meu-projeto.

2. Fork
Um **fork** é uma cópia de um repositório remoto para a sua conta, permitindo que você faça alterações sem afetar o repositório original. Muito usado em projetos open source.

Exemplo prático:

- Vá até um repositório no GitHub e clique em "Fork".
- Clone o repositório para sua máquina:bash
- CopiarEditar
- git clone https://github.com/seu-usuario/meu-fork.git cd meu-fork
- Adicione o repositório original como remoto:bash
- CopiarEditar
- git remote add upstream https://github.com/original-autor/repo.git

3. Branch
Um **branch** (ramo) é uma linha de desenvolvimento separada da principal (main ou master). Ele permite que você trabalhe em novas funcionalidades sem afetar o código principal.

Exemplo prático:
Criando e mudando para um novo branch:
git branch nova-funcionalidade
git checkout nova-funcionalidade # Ou diretamente: git checkout -b nova-funcionalidade

Listando os branches:
git branch

4. Commit
O **commit** é um registro das alterações feitas nos arquivos. Ele serve como um ponto de restauração no histórico do projeto.

Exemplo prático:
Adicionando arquivos ao commit e registrando as mudanças:

git add arquivo.txt git commit -m "Adicionando arquivo de exemplo"

Verificando o histórico de commits:
git log --oneline

5. Merge
O **merge** combina as alterações de um branch com outro. É usado para integrar código desenvolvido em um branch secundário de volta ao principal.

Exemplo prático:
Voltando para a branch main e fazendo o merge:
git checkout main git merge nova-funcionalidade

Isso incorpora as mudanças do branch nova-funcionalidade na main.

6. Conflitos

Os **conflitos** acontecem quando duas pessoas alteram a mesma parte do código em branches diferentes. O Git não consegue decidir automaticamente qual versão usar e exige intervenção manual.

Exemplo prático:

- Se ao tentar um merge você receber um erro de conflito:
  Auto-merging arquivo.txt
  CONFLICT (content): Merge conflict in arquivo.txt

- Abra o arquivo e resolva o conflito, escolhendo qual versão do código manter. O Git marcará as seções conflitantes assim:txt

```
<<<<<<< HEAD
Código da branch atual
=======
Código da outra Branch
>>>>>>> nova-funcionalidade
```

- Após resolver o conflito, faça um novo commit:
  git add arquivo.txt git commit -m "Resolvendo conflito no arquivo.txt"

Conclusão

O Git é uma ferramenta poderosa para controle de versão, permitindo colaboração eficiente em projetos. Esses conceitos básicos — repositórios, forks, branches, commits, merges e resolução de conflitos — são essenciais para trabalhar com Git em qualquer ambiente profissional.